

Integración Commit-Docs — Contrato del Webhook

Audiencia: equipo que implementa el agente externo (Bitbucket Pipelines / GitHub Actions / cualquier IA que documenta commits) y debe enviar la documentación generada al backend de Kull SpA.

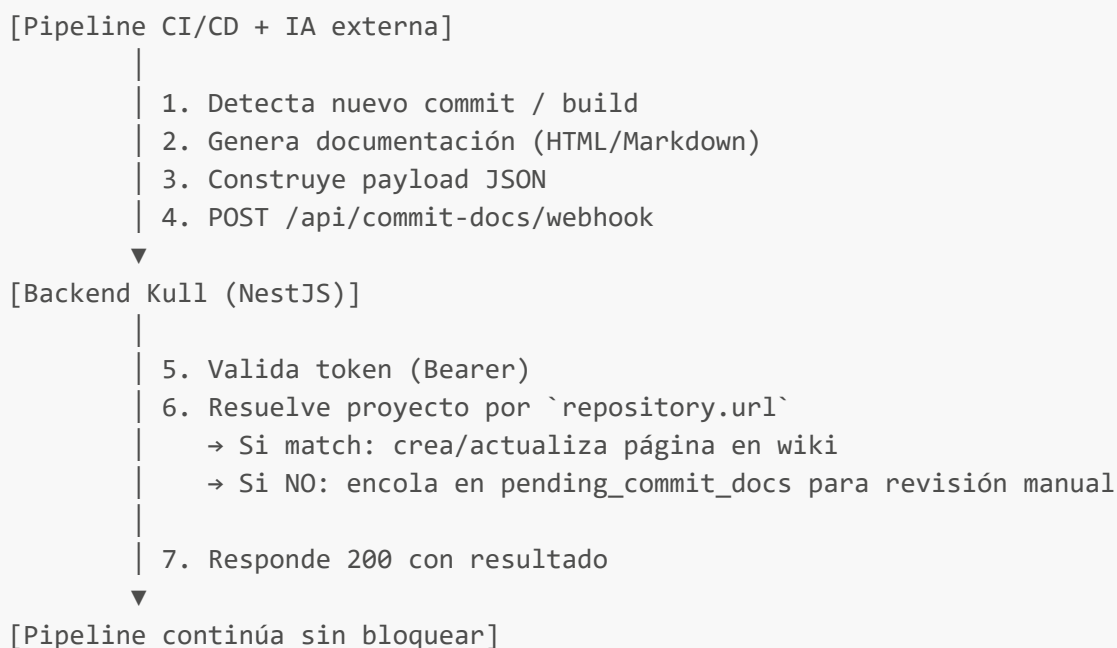
Endpoint productivo: <https://api-manage.kull.cl/api/commit-docs/webhook>

Última actualización: 2026-05-18 · Versión 1.0

1. Resumen

El backend de Kull expone un endpoint REST que recibe la documentación generada por una IA externa para cada commit/release y la publica **automáticamente** como página en el wiki interno del proyecto al que pertenece el repositorio (categoría **changelog**).

Flujo de alto nivel



Garantías del endpoint

- **Idempotente:** reenviar el mismo (**repo**, **commitSha**) actualiza la página existente en lugar de duplicarla.
- **No bloqueante:** nunca devuelve 4xx por "repo no asociado". Si no hay match, la entrada queda en cola para revisión manual y se responde **200** con **status: 'queued'**.
- **Truncado defensivo:** diffs > 50 KB y test output > 20 KB se recortan antes de persistir, para no romper la BD.

- **Auditable:** cada llamada queda con timestamp `createdAt`, payload completo persistido y trazabilidad por commit SHA.

2. Autenticación

Todas las llamadas requieren el header:

```
Authorization: Bearer <COMMIT_DOCS_WEBHOOK_TOKEN>
```

- Token compartido (32 bytes / 64 caracteres hex).
- Validación con comparación en tiempo constante (resistente a timing attacks).
- Sin token → **401 Unauthorized**.
- Token inválido → **401 Unauthorized**.
- Si el servidor no tiene el token configurado, **todas** las llamadas son rechazadas (fail-safe).

El token se entrega por canal seguro (NO está en este documento ni en ningún archivo del repo). Pídelo al admin de Kull o tómallo del archivo `.env` del backend.

Cómo guardarlo en Bitbucket Pipelines

Repository settings → Pipelines → Repository variables → Add:

Name	Value	Secured
KULL_COMMIT_DOCS_TOKEN	(el token)	☒ Sí
KULL_COMMIT_DOCS_URL	https://api-manage.kull.cl/api/commit-docs/webhook	No

Cómo guardarlo en GitHub Actions

Settings → Secrets and variables → Actions → New repository secret:

Name	Value
KULL_COMMIT_DOCS_TOKEN	(el token)

3. Endpoint principal — **POST** /webhook

Request

```
POST /api/commit-docs/webhook HTTP/1.1
Host: api-manage.kull.cl
```

Authorization: Bearer <COMMIT_DOCS_WEBHOOK_TOKEN>
Content-Type: application/json

```
{ ...payload... }
```

Body (payload JSON)

Compatible con el formato de Bitbucket Pipelines /

Automation-scripts-bitbucket. La única regla dura es que

repository.url debe venir; todo lo demás es opcional y se omite del documento si falta.

```
{
  // — Metadata de la fuente (opcional, queda como traza) —————
  "source": "bitbucket-pipelines",
  "generatedBy": "Automation-scripts-bitbucket",
  "generatedAt": "2026-05-18T15:30:00.000Z",
  "date": "2026-05-18",

  // — Identificación del proyecto (opcional pero recomendado) —————
  // Se usa solo como hint para la asignación manual cuando el repo no
  // está asociado. No reemplaza el matching por `repository.url`.
  "project": {
    "key": "DDS",
    "slug": "automation-tests",
    "name": "automation-tests"
  },

  // — Repositorio (REQUERIDO `url`) —————
  "repository": {
    "url": "https://bitbucket.org/automation-confluence/automation-tests", //
    ⚠ obligatorio
    "fullName": "automation-confluence/automation-tests",
    "slug": "automation-tests",
    "targetPath": "/opt/atlassian/pipelines/agent/build"
  },

  // — Release / commit (recomendado: pasar siempre `commit`) —————
  "release": {
    "title": "[BROKEN] Release 2026-05-18 - main - e60adbb",
    "status": "broken", // ok | broken | warning | failed | success
    | passed
    "branch": "main",
    "commit": "e60adbb1234567890", // ★ SHA completo. Si está, habilita
    idempotencia.
    "commitShort": "e60adbb"
  },

  // — Documentación generada por la IA —————
  // Pasa html o markdown (al menos uno). Si pasas ambos, se usa markdown.
```

```

"content": {
  "html":      "<h1>Release 2026-05-18 - main</h1>...", // se embebe tal cual
  "markdown":  "# Release 2026-05-18 - main\n..."      // alternativa
preferida
},

// — Cambios (opcional, se muestran en el doc generado) —————
"changes": {
  "commits":    "e60adbb - feat: add healthcheck metadata (Demo Dev)",
  "diffStat":   "ticket-api-demo/src/app.js | 4 +++-",
  "diffSummary": "diff --git a/ticket-api-demo/src/app.js b/ticket-api-
demo/src/app.js\n..."
},

// — Tests (opcional) —————
"tests": {
  "status": "failed", // ok | passed | failed | warning | etc.
  "failed": true,
  "output": "FAIL tests/tickets.test.js\nInternal Ticket API › GET /health
..."
},

// — Routing (opcional, informativo) —————
"routing": {
  "wikiPath": "/proyectos/automation-tests/wiki",
  "projectSlugSource": "repository.slug"
},

// — Override avanzado (opcional) —————
// Si conoces el ID canónico del proyecto en Kull, lo puedes mandar
// directo y se salta el matching por repo. Útil para monorepos o
// cuando un mismo repo debe documentar en N proyectos.
"projectId": "proj-1772830766493-7lnkw5pa"
}

```

Campos: detalle

Campo	Tipo	Obligatorio	Notas
<code>repository.url</code>	string	Sí	Validación falla con 400 si falta. Es la clave del matching contra <code>Project.repos</code> .
<code>release.commit</code>	string	No, pero recomendado	Si está, habilita idempotencia: re-recibir el mismo SHA actualiza la página en vez de duplicar.
<code>release.status</code>	string	No	Determina el ícono del título:  broken/failed/error ·  warning ·  ok/success/passed ·  unknown.
<code>release.title</code>	string	No	Si está, se usa como título de la página. Si no, se construye uno con <code>branch + commitShort</code> .

Campo	Tipo	Obligatorio	Notas
<code>content.html</code> o <code>content.markdown</code>	string	No, pero útil	El contenido principal del doc. Si ambos, gana <code>markdown</code> .
<code>changes.diffSummary</code>	string	No	Se trunca a 50k chars (resto se omite con marca explícita).
<code>tests.output</code>	string	No	Se trunca a 20k chars.
<code>projectId</code>	string	No	Override directo del matching por repo. Útil para monorepos.

Responses

200 OK — Página creada/actualizada

Cuando el repo matcheó con un `Project.repos` y se creó/actualizó la página en el wiki:

```
{
  "success": true,
  "status": "created", // o "updated" en re-recepción
  "projectId": "proj-1772830766493-7lnkvv5pa",
  "projectName": "Addons Logica B2BKing",
  "doc": {
    "id": 142,
    "title": "🌀 Release 2026-05-18 - main - e60adbb",
    "slug": "release-2026-05-18-main-e60adbb",
    "url": "/proyectos/proj-1772830766493-7lnkvv5pa/wiki?doc=142",
    "category": "changelog"
  },
  "matchedBy": "repo", // o "projectId" si pasaste
  override
  "commitSha": "e60adbb1234567890"
}
```

200 OK — Encolado para revisión manual

Cuando el repo NO matcheó. **Esto NO es un error.** El payload queda guardado y un admin puede asignarlo al proyecto correcto desde `/commit-docs` en el dashboard.

```
{
  "success": true,
  "status": "queued",
  "pendingId": 27,
  "reason": "No se encontró ningún proyecto cuyo campo 'repos' contenga
```

```
\ "https://bitbucket.org/foo/bar\" ni \"foo/bar\".\",
  \"repositoryUrl\": \"https://bitbucket.org/foo/bar\",
  \"commitSha\": \"e60adbb1234567890\"
}
```

400 Bad Request — Payload inválido

Falta `repository.url` o tipo incorrecto:

```
{
  \"statusCode\": 400,
  \"message\": [
    \"repository.url es obligatorio. La IA externa debe enviar siempre la URL
    del repositorio para que podamos resolver el proyecto destino.\"
  ],
  \"error\": \"Bad Request\"
}
```

401 Unauthorized

Sin header, header malformado o token inválido:

```
{
  \"statusCode\": 401,
  \"message\": \"Token inválido\",
  \"error\": \"Unauthorized\"
}
```

500 Internal Server Error

Solo si la BD está caída o algo inesperado falló. La IA puede reintentar con backoff exponencial.

4. Endpoints auxiliares

GET /api/commit-docs/health

Health check liviano. Útil para verificar conectividad y token antes de empezar a mandar tráfico real desde el pipeline.

```
GET /api/commit-docs/health
Authorization: Bearer <TOKEN>
```

Respuesta:

```
{ "ok": true, "service": "commit-docs", "timestamp": "2026-05-18T19:21:59.093Z"
}
```

GET /api/commit-docs/test-match

Dry-run del matching de repo. NO crea nada. Devuelve qué proyecto matchearía con el repo dado. Indispensable para configurar el campo **Project.repos** correctamente antes de mandar tráfico real.

```
GET /api/commit-docs/test-match?repo=https://bitbucket.org/owner/repo
Authorization: Bearer <TOKEN>
```

Respuesta cuando hay match:

```
{
  "found": true,
  "input": "https://bitbucket.org/owner/repo",
  "project": {
    "id": "proj-1772830766493-7lnkvv5pa",
    "name": "Addons Logica B2BKing",
    "clientId": "client-1772830600314-831u1bgzw",
    "repos": ["https://bitbucket.org/owner/repo"]
  }
}
```

Cuando no hay match:

```
{ "found": false, "input": "https://bitbucket.org/owner/repo", "project": null
}
```

5. Matching de repositorios

El backend normaliza ambos lados antes de comparar:

- Lowercase.
- Quita `https://` / `http://` / `git@host:.`
- Quita sufijo `.git`.
- Quita trailing slash.

Y luego compara en este orden:

1. **URL completa normalizada** (match exacto).
2. **owner/repo** (fullName, últimos 2 segmentos).
3. **slug** (último segmento) — **solo si es único** en todo el dataset. Si el slug colisiona entre varios proyectos, no se asigna (para evitar falsos positivos con nombres genéricos tipo **api**, **frontend**, **web**).

Implicancia para el agente externo

Manda siempre la URL completa. Es la opción más segura y rápida.
Solo en último caso usa **slug** o **fullName**, y validá primero con **/test-match** que no haya colisión.

6. Idempotencia y reintentos

Cuándo se actualiza vs se crea

El backend usa **sourceTranscriptionId = "commit:<repoFullName>:<sha>"** como llave de idempotencia. Esto significa:

Escenario	Comportamiento
Primera vez que llega un (repo , sha)	Crea nueva página → status: "created"
Segunda vez (mismo repo , mismo sha)	Actualiza la página existente → status: "updated"
Sin release.commit en el payload	Crea siempre nueva página (sin idempotencia)

Estrategia de reintentos recomendada para el agente

Si la llamada falla con **5xx** o timeout:

```
Reintento 1: esperar 5s
Reintento 2: esperar 15s
Reintento 3: esperar 60s
Después: marcar como fallida y notificar al equipo
```

Las idempotencias protegen contra duplicados aunque uses retries agresivos, **siempre que mandes release.commit**.

No reintentar con **4xx** (es un problema de payload o token; reintentarlo no lo va a arreglar).

7. Ejemplos de implementación

7.1 — cURL (test rápido)


```
#!/bin/bash
set -e

TOKEN="<COMMIT_DOCS_WEBHOOK_TOKEN>"
API="https://api-manage.kull.cl"

# Health check
curl -fsSL \
  -H "Authorization: Bearer $TOKEN" \
  "$API/api/commit-docs/health"

# Test de matching
curl -fsSL \
  -H "Authorization: Bearer $TOKEN" \
  "$API/api/commit-docs/test-match?repo=https://bitbucket.org/owner/repo"

# Envío real
curl -fsSL -X POST "$API/api/commit-docs/webhook" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d @payload.json
```

7.2 — Bash script para Bitbucket Pipelines

Asume que la IA ya generó el `payload.json` en el step anterior.

```
#!/bin/bash
# scripts/send-commit-docs.sh
set -euo pipefail

PAYLOAD_FILE="${1:-payload.json}"

if [ ! -f "$PAYLOAD_FILE" ]; then
  echo "✗ No existe $PAYLOAD_FILE"
  exit 1
fi

API_URL="${KULL_COMMIT_DOCS_URL:-https://api-manage.kull.cl/api/commit-docs/webhook}"
TOKEN="${KULL_COMMIT_DOCS_TOKEN:?Variable KULL_COMMIT_DOCS_TOKEN no definida}"

# Reintento con backoff: 3 intentos
for attempt in 1 2 3; do
  echo "🚀 Intento $attempt: enviando commit-doc a $API_URL"

  HTTP_CODE=$(curl -sS -w "%{http_code}" -o /tmp/commit_docs_response.json \
    -X POST "$API_URL" \
    -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" \
    -d @"$PAYLOAD_FILE")
```

```

case "$HTTP_CODE" in
  200)
    echo "✅ Webhook OK:"
    cat /tmp/commit_docs_response.json | jq .
    exit 0
    ;;
  400|401)
    echo "❌ Error de cliente ($HTTP_CODE) – no reintentar:"
    cat /tmp/commit_docs_response.json
    exit 1
    ;;
  *)
    echo "⚠️ Error $HTTP_CODE. Reintentando en $((attempt * 5))s..."
    sleep $((attempt * 5))
    ;;
esac
done

echo "❌ Falló tras 3 intentos"
exit 1

```

Uso:

```

chmod +x scripts/send-commit-docs.sh
./scripts/send-commit-docs.sh payload.json

```

7.3 — `bitbucket-pipelines.yml` completo

```

image: node:20

pipelines:
  branches:
    main:
      - step:
          name: Build & test
          script:
            - npm ci
            - npm run build
            - npm test 2>&1 | tee /tmp/test_output.txt || true
          artifacts:
            - dist/**
            - /tmp/test_output.txt

      - step:
          name: Generar documentación con IA
          script:
            - npm install -g your-ai-doc-generator # tu IA

```

```

- your-ai-doc-generator \
  --commit "$BITBUCKET_COMMIT" \
  --branch "$BITBUCKET_BRANCH" \
  --output-html /tmp/ai-doc.html
# Construir el payload (usa jq para JSON seguro)
- |
  jq -n \
    --arg url
"https://bitbucket.org/${BITBUCKET_REPO_FULL_NAME}" \
    --arg full    "$BITBUCKET_REPO_FULL_NAME" \
    --arg slug    "$BITBUCKET_REPO_SLUG" \
    --arg commit  "$BITBUCKET_COMMIT" \
    --arg short   "${BITBUCKET_COMMIT:0:7}" \
    --arg branch  "$BITBUCKET_BRANCH" \
    --arg date    "$(date -u +%Y-%m-%d)" \
    --arg now     "$(date -u +%Y-%m-%dT%H:%M:%S.000Z)" \
    --arg html    "$(cat /tmp/ai-doc.html)" \
    --arg diff    "$(git log -1 --stat --format=' ' || echo ' ')" \
    --arg tests   "$(cat /tmp/test_output.txt || echo ' ')" \
    '{
      source: "bitbucket-pipelines",
      generatedBy: "ai-doc-generator",
      generatedAt: $now,
      date: $date,
      repository: { url: $url, fullName: $full, slug: $slug },
      release: {
        title: "Release \($date) - \($branch) - \($short)",
        branch: $branch,
        commit: $commit,
        commitShort: $short,
        status: "ok"
      },
      content: { html: $html },
      changes: { diffStat: $diff },
      tests: { output: $tests }
    }' > /tmp/payload.json

- step:
  name: Enviar documentación a Kull
  script:
    - bash scripts/send-commit-docs.sh /tmp/payload.json

```

7.4 — GitHub Actions workflow

```

name: Send commit-docs to Kull

on:
  push:
    branches: [main]

```

```

jobs:
  send-commit-docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 2

      - name: Generate AI doc
        id: ai
        run: |
          # Tu generador de documentación
          your-ai-doc-generator \
            --commit "${{ github.sha }}" \
            --output-html /tmp/ai-doc.html

      - name: Build payload
        run: |
          jq -n \
            --arg url      "https://github.com/${{ github.repository }}" \
            --arg full      "${{ github.repository }}" \
            --arg slug      "${{ github.event.repository.name }}" \
            --arg commit    "${{ github.sha }}" \
            --arg short     "$(echo ${{ github.sha }} | cut -c1-7)" \
            --arg branch    "${{ github.ref_name }}" \
            --arg now       "$(date -u +%Y-%m-%dT%H:%M:%S.000Z)" \
            --arg date      "$(date -u +%Y-%m-%d)" \
            --arg html      "$(cat /tmp/ai-doc.html)" \
            '{
              source: "github-actions",
              generatedBy: "ai-doc-generator",
              generatedAt: $now,
              date: $date,
              repository: { url: $url, fullName: $full, slug: $slug },
              release: {
                title: "Release \($date) - \($branch) - \($short)",
                branch: $branch,
                commit: $commit,
                commitShort: $short,
                status: "ok"
              },
              content: { html: $html }
            }' > /tmp/payload.json

      - name: Send to Kull
        env:
          KULL_TOKEN: ${ secrets.KULL_COMMIT_DOCS_TOKEN }
        run: |
          curl -fsSL -X POST https://api-manage.kull.cl/api/commit-docs/webhook \

            -H "Authorization: Bearer $KULL_TOKEN" \
            -H "Content-Type: application/json" \
            -d @/tmp/payload.json

```

7.5 — Node.js (TypeScript)

```
import fetch from 'node-fetch';

interface CommitDocsPayload {
  source?: string;
  generatedBy?: string;
  generatedAt?: string;
  date?: string;
  project?: { key?: string; slug?: string; name?: string };
  repository: { url: string; fullName?: string; slug?: string; targetPath?:
string };
  release: {
    title?: string;
    status?: string;
    branch?: string;
    commit?: string;
    commitShort?: string;
  };
  content?: { html?: string; markdown?: string };
  changes?: { commits?: string; diffStat?: string; diffSummary?: string };
  tests?: { status?: string; failed?: boolean; output?: string };
  routing?: { wikiPath?: string; projectSlugSource?: string };
  projectId?: string;
}

interface CommitDocsResponse {
  success: boolean;
  status: 'created' | 'updated' | 'queued';
  projectId?: string;
  projectName?: string;
  doc?: { id: number; title: string; slug: string; url: string; category:
string | null };
  matchedBy?: 'projectId' | 'repo';
  commitSha: string | null;
  pendingId?: number;
  reason?: string;
  repositoryUrl?: string;
}

async function sendCommitDoc(payload: CommitDocsPayload):
Promise<CommitDocsResponse> {
  const token = process.env.KULL_COMMIT_DOCS_TOKEN;
  const url = process.env.KULL_COMMIT_DOCS_URL
    ?? 'https://api-manage.kull.cl/api/commit-docs/webhook';

  if (!token) throw new Error('KULL_COMMIT_DOCS_TOKEN no definido');

  // Reintento con backoff
```

```

const delays = [0, 5_000, 15_000, 60_000];

for (let attempt = 0; attempt < delays.length; attempt++) {
  if (delays[attempt] > 0) await new Promise(r => setTimeout(r,
delays[attempt]));

  const res = await fetch(url, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(payload),
  });

  if (res.status === 200) {
    return await res.json() as CommitDocsResponse;
  }

  // Errores de cliente: no reintentar
  if (res.status === 400 || res.status === 401) {
    const errBody = await res.text();
    throw new Error(`Error ${res.status}: ${errBody}`);
  }

  console.warn(`Intento ${attempt + 1} falló con ${res.status},
reintentando...`);
}

throw new Error('Falló tras 3 reintentos');
}

// Uso
(async () => {
  const result = await sendCommitDoc({
    source: 'my-ci',
    repository: { url: 'https://github.com/owner/repo' },
    release: {
      commit: 'abc123def456',
      commitShort: 'abc123d',
      branch: 'main',
      status: 'ok',
      title: 'Release deploy-2026-05-18',
    },
    content: {
      markdown: '# Cambios\n\n- Fix bug en login\n- Mejora performance',
    },
  });

  console.log(result);
  if (result.status === 'queued') {
    console.warn(`⚠ Repo no asociado. Pendiente #${result.pendingId} en
revisión.`);
  }
}

```

```

    } else {
        console.log(`✅ Doc ${result.doc?.id} ${result.status}:
${result.doc?.url}`);
    }
}
})();

```

7.6 — Python (con requests)

```

import os
import time
import requests
from typing import TypedDict, Optional

API = os.environ.get(
    "KULL_COMMIT_DOCS_URL",
    "https://api-manage.kull.cl/api/commit-docs/webhook",
)
TOKEN = os.environ["KULL_COMMIT_DOCS_TOKEN"]

def send_commit_doc(payload: dict, max_attempts: int = 3) -> dict:
    """Envía un commit-doc al webhook de Kull con reintento exponencial."""
    headers = {
        "Authorization": f"Bearer {TOKEN}",
        "Content-Type": "application/json",
    }

    backoff = [0, 5, 15]
    last_err = None

    for attempt in range(max_attempts):
        if backoff[attempt]:
            time.sleep(backoff[attempt])

        try:
            r = requests.post(API, json=payload, headers=headers, timeout=30)
        except requests.RequestException as e:
            last_err = e
            print(f"⚠️ Intento {attempt + 1} excepción: {e}")
            continue

        if r.status_code == 200:
            return r.json()
        if r.status_code in (400, 401):
            raise RuntimeError(f"Error {r.status_code}: {r.text}")

        last_err = RuntimeError(f"HTTP {r.status_code}: {r.text}")
        print(f"⚠️ Intento {attempt + 1} falló: {last_err}")

    raise last_err or RuntimeError("Falló tras reintentos")

```

```

if __name__ == "__main__":
    payload = {
        "source": "my-ci-python",
        "generatedAt": "2026-05-18T20:00:00.000Z",
        "repository": {
            "url": "https://github.com/owner/repo",
            "fullName": "owner/repo",
        },
        "release": {
            "title": "Release v1.2.3",
            "branch": "main",
            "commit": "abc123def456",
            "commitShort": "abc123d",
            "status": "ok",
        },
        "content": {
            "markdown": "# v1.2.3\n\n Bug fix\n- Feature X",
        },
    }

    result = send_commit_doc(payload)
    print("✅", result)

```

8. Checklist de implementación

Antes de poner el agente en producción:

- ☐ El token `KULL_COMMIT_DOCS_TOKEN` está configurado como secret/variable segura en el pipeline.
- ☐ Probaste `GET /api/commit-docs/health` con el token y devuelve `200`.
- ☐ Probaste `GET /api/commit-docs/test-match?repo=<tu-repo>` y confirmaste que matchea con el proyecto correcto (si no, pídele al admin que agregue el repo en `/proyectos`).
- ☐ El payload mínimo contiene `repository.url` y `release.commit`.
- ☐ El pipeline maneja reintentos con backoff para 5xx (no para 4xx).
- ☐ El pipeline NO falla si la respuesta es `status: "queued"` — eso es normal hasta que el admin asocie el repo.
- ☐ Si tu pipeline corre muchos commits seguidos, validar que la idempotencia funcione (re-disparar el mismo commit no genera duplicados).
- ☐ El HTML/Markdown que envía la IA está sanitizado (no scripts inyectados).

9. FAQ / Troubleshooting

"Me responde 401 con un token que creo correcto"

- Verifica que el header sea exactamente `Authorization: Bearer <token>` (con la palabra `Bearer` y espacio).
- Verifica que no haya espacios o saltos de línea al final del token (cuidado con `echo` que agrega `\n`; usa `printf` o `echo -n`).
- Pide al admin que confirme el valor del token en el `.env` del servidor.

"Me responde 400 con 'repository.url es obligatorio'"

- El campo `repository.url` falta o está vacío. Es el único campo estrictamente obligatorio.

"Me responde 200 con 'status: queued' siempre"

- El repo no está asociado al proyecto. Soluciones:
 1. **Recomendado:** el admin agrega el repo al campo "Repositorios asociados" del proyecto desde `/proyectos`.
 2. **Workaround:** el agente externo pasa `projectId` explícito en el payload para saltarse el matching.

"¿Cómo veo qué proyectos están configurados con qué repos?"

- Desde el dashboard interno (`/proyectos`) — campo "Repositorios asociados".
- Desde el endpoint de test: `GET /api/commit-docs/test-match?repo=<url>`.

"¿Las páginas creadas son visibles al cliente?"

- **No por default.** Se crean con `visibleToClient: false`. Un admin puede marcarla como visible desde el wiki si quiere compartirla con el cliente.

"¿Qué pasa si mando el mismo commit dos veces?"

- Se actualiza la página existente (idempotencia por SHA). No genera duplicados. **Requiere que mandes `release.commit`** — si no lo mandas, cada llamada crea una página nueva.

"¿Qué pasa con commits muy grandes (mucho diff o test output)?"

- `changes.diffSummary` se trunca a 50 KB.
- `tests.output` se trunca a 20 KB.
- El truncado aparece como comentario explícito en el doc final, no silenciosamente.

"¿Puedo usar el endpoint desde localhost / dev?"

- Sí, apuntando a `http://localhost:3000/api/commit-docs/webhook` con el mismo token que esté en el `.env` local del backend.

"¿Hay rate limiting?"

- No hay rate limit por token específico, pero el backend está detrás de un balanceador estándar. Si vas a mandar > 100 req/min sostenidos, avisa al admin para revisar.
-

10. Contacto

- **Admin del backend:** sebastian@kull.cl
- **Repo backend:** <https://github.com/sbriceno/jiraNotion>
- **Repo frontend (dashboard):** <https://github.com/sbriceno/dashboard-rentabilidad>
- **Documentación interna:** [docs/arquitectura.md](#) (sección 11.b)